

ALUNO: João Ricardo Barbiero de Souza | RA: 24000499-2

ENTRADA: resultado = (i + j)/100;

LEXEMA	TOKEN

resultado	IDENTIFICADOR
=	ATRIBUICAO
(	ABRE_PARENTESES
i	IDENTIFICADOR
+	SOMA
j	IDENTIFICADOR
)	FECHA_PARENTESES
/	DIVISAO
100	INTEIRO
;	PONTO_E_VIRGULA

ALUNO: João Ricardo Barbiero de Souza | RA: 24000499-2

=== TESTE A - entrada compacta ===

ENTRADA: int total=valor+10;

LEXEMA	CLASSE	TOKEN

int	LETRA	PALAVRA_RESERVADA
total	LETRA	IDENTIFICADOR
=	SIMBOLO	ATRIBUICAO
valor	LETRA	IDENTIFICADOR
+	SIMBOLO	SOMA
10	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE B - espaços e comentário ===

ENTRADA: int total = valor + 10; // este comentario sera ignorado

LEXEMA	CLASSE	TOKEN

int	LETRA	PALAVRA_RESERVADA
total	LETRA	IDENTIFICADOR
=	SIMBOLO	ATRIBUICAO
valor	LETRA	IDENTIFICADOR
+	SIMBOLO	SOMA
10	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA
// este comentario sera ignorado	COMENTARIO	COMENTARIO_IGNOREADO

=== TESTE C - reservadas e identificadores ===

ENTRADA: int inteiro = 0; while (inteiro < intValor) inteiro = inteiro + 1;

LEXEMA	CLASSE	TOKEN

int	LETRA	PALAVRA_RESERVADA
inteiro	LETRA	IDENTIFICADOR
=	SIMBOLO	ATRIBUICAO
0	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA
while	LETRA	PALAVRA_RESERVADA
(	SIMBOLO	ABRE_PARENTESES
inteiro	LETRA	IDENTIFICADOR
<	SIMBOLO	MENOR_QUE
intValor	LETRA	IDENTIFICADOR
)	SIMBOLO	FECHA_PARENTESES
inteiro	LETRA	IDENTIFICADOR
=	SIMBOLO	ATRIBUICAO
inteiro	LETRA	IDENTIFICADOR
+	SIMBOLO	SOMA
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE C2 - identificador personalizado ===

ENTRADA: int nota_jr = 10;

LEXEMA	CLASSE	TOKEN

int	LETRA	PALAVRA_RESERVADA
nota_jr	LETRA	IDENTIFICADOR
=	SIMBOLO	ATRIBUICAO
10	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE D - caractere inválido ===

ENTRADA: int valor# = 1;

LEXEMA	CLASSE	TOKEN

int	LETRA	PALAVRA_RESERVADA
valor	LETRA	IDENTIFICADOR
#	DESCONHECIDO	ERRO_LEXICO
=	SIMBOLO	ATRIBUICAO
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

```
Main.java:8: error: ';' expected
    int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
                        ^
Main.java:10: error: ')' expected
    if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
                   ^
Main.java:11: error: unclosed string literal
    System.out.println("Programa corrigido!); // CORRIGIR 3: feche o texto entre aspas
                      ^
3 errors
```

ALUNO: João Ricardo Barbiero de Souza | RA: 24000499-2

Programa corrigido!

Resultado: 30

ALUNO: João Ricardo Barbiero de Souza | RA: 24000499-2

ENTRADA: 2 + 3 * 4

```
ENTRANDO em <expr> | lookahead = 2
  ENTRANDO em <term> | lookahead = 2
    ENTRANDO em <factor> | lookahead = 2
      Numero reconhecido: 2
      Novo lookahead: +
    SAINDO de <factor> | lookahead = +
  SAINDO de <term> | lookahead = +
  Encontrado operador +
  Novo lookahead: 3
  ENTRANDO em <term> | lookahead = 3
    ENTRANDO em <factor> | lookahead = 3
      Numero reconhecido: 3
      Novo lookahead: *
    SAINDO de <factor> | lookahead = *
  Encontrado operador *
  Novo lookahead: 4
  ENTRANDO em <factor> | lookahead = 4
    Numero reconhecido: 4
    Novo lookahead: $FIM$
  SAINDO de <factor> | lookahead = $FIM$
  SAINDO de <term> | lookahead = $FIM$
  SAINDO de <expr> | lookahead = $FIM$
```

ENTRADA ACEITA

RESULTADO DA EXPRESSAO: 14

ALUNO: João Ricardo Barbiero de Souza | RA: 24000499-2

ENTRADA: 2 + * 4

ENTRANDO em <expr> | lookahead = 2
ENTRANDO em <term> | lookahead = 2
ENTRANDO em <factor> | lookahead = 2
Numero reconhecido: 2
Novo lookahead: +
SAINDO de <factor> | lookahead = +
SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: *
ENTRANDO em <term> | lookahead = *
ENTRANDO em <factor> | lookahead = *

ERRO SINTATICO: era esperado um numero ou '('; lookahead = *

ALUNO: João Ricardo Barbiero de Souza | RA: 24000499-2

ENTRADA: id+id*id

PILHA	ENTRADA	ACAO

\$ 0	id+id*id\$	SHIFT id -> estado 5
\$ 0 id 5	+id*id\$	REDUCE F -> id
\$ 0 F 3	+id*id\$	REDUCE T -> F
\$ 0 T 2	+id*id\$	REDUCE E -> T
\$ 0 E 1	+id*id\$	SHIFT + -> estado 6
\$ 0 E 1 + 6	id*id\$	SHIFT id -> estado 5
\$ 0 E 1 + 6 id 5	*id\$	REDUCE F -> id
\$ 0 E 1 + 6 F 3	*id\$	REDUCE T -> F
\$ 0 E 1 + 6 T 9	*id\$	SHIFT * -> estado 7
\$ 0 E 1 + 6 T 9 * 7	id\$	SHIFT id -> estado 5
\$ 0 E 1 + 6 T 9 * 7 id 5	\$	REDUCE F -> id
\$ 0 E 1 + 6 T 9 * 7 F 10	\$	REDUCE T -> T * F
\$ 0 E 1 + 6 T 9	\$	REDUCE E -> E + T
\$ 0 E 1	\$	ACCEPT